# VIBETUNE: VOICE AND MOOD BASED MUSIC PLAYER

*A PROJECT REPORT*

*Submitted by*

**DEBJYOTI**

**MITRA**

**Reg No-230301120542**

*in partial fulfilment for the award of the degree of*

**BACHELOR OF TECHNOLOGY**

*in*

**COMPUTER SCIENCE AND ENGINEERING**




Centurion
UNIVERSITY
*Shaping Lives...*
*Empowering Communities...*

**DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING**

**SCHOOL OF ENGINEERING AND TECHNOLOGY**

**BHUBANESWAR CAMPUS**

*CENTURION UNIVERSITY OF TECHNOLOGY AND MANAGEMENT*

*ODISHA*

**November, 2026**

**DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING**
**SCHOOL OF ENGINEERING AND TECHNOLOGY**
**BHUBANESWAR CAMPUS**

## BONAFIDE CERTIFICATE

Certified that this project report "VIBETUNE: VOICE AND MOOD BASED MUSIC PLAYER" is the Bonafide work of "DEBJYOTI MITRA" (Reg No: 230301120542) who carried out the project work under my supervision. This is to further certify to the best of my knowledge, that this project has not been carried out earlier in this institute and the university.

Under the Supervision of
**(Mr. Chittaranjan Behera)**
Assistant Professor of CSE, SoET

*Certified that the above-mentioned project has been duly carried out as per the norms of the college and statutes of the university*

**SIGNATURE**
**HOD/DEAN**

**DEPARTMENT SEAL**

## DECLARATION

I hereby declare that the project entitled “VIBETUNE: VOICE AND MOOD BASED MUSIC PLAYER” submitted for the “MINOR PROJECT -II” of 7th semester B. Tech in Computer Science and Engineering is my original work and the project has not formed the basis for the award of any Degree / Diploma or any other similar titles in any other University / Institute.

**Name of the Student:**

**Signature of the Student:**

**Registration No: 230301120542**

**Place: Bhubaneswar**

**Date:**

# ACKNOWLEDGEMENTS

I wish to express my profound and sincere gratitude to Mr. Chittaranjan Behera, Department of Computer Science and Engineering, SoET, Bhubaneswar Campus, who guided me into the intricacies of this project nonchalantly with matchless magnanimity.

I thank Prof. Rahul Roy, Head of the Dept. of Department of Computer Science and Engineering, SoET, Bhubaneswar Campus and Dr. Swarnaprabha Jena, Dean, School of Engineering and Technology, Bhubaneswar Campus for extending their support during the course of this investigation.

I would be failing in my duty if I don't acknowledge the cooperation rendered during various stages of Networking model design, CISCO architecture configuration, and simulation analysis by Mr. Chittaranjan Behera.

I am highly grateful to Mr. Chittaranjan Behera who evinced keen interest and invaluable support in the progress and successful completion of my project work.

I am indebted to my parents for their constant encouragement, co-operation and help. Words of gratitude are not enough to describe the accommodation and fortitude which they have shown throughout my endeavor.

**Name of the Student:**

**Signature of the Student:**

**Registration No: 230301120542**

**Place: Bhubaneswar**

**Date:**

## Abstract

VibeTune is a browser-based music player that combines a personal audio library with voice and text commands. This report describes the application visible in the public repository: a Flask server renders the player, discovers audio files stored in mood-named folders, and exposes a command-classification endpoint. In the browser, the Web Speech API can convert spoken English into text, while a scikit-learn pipeline classifies natural-language requests into playback, volume, navigation, and mood intents. The frontend also provides conventional player controls, favorites, custom playlists, search, theme preferences, and an optional camera expression-estimation feature.

The system is designed to make everyday playback convenient without requiring a paid speech-recognition service or cloud credentials for its core command workflow. Mood labels represent the user's expressed intent, such as asking for calm or focus music; they are not inferred from vocal tone. The optional camera feature is separate: face-api.js estimates a facial expression locally in the browser and maps it to a category. The repository does not include music recordings, so a user supplies lawful audio files in supported folders.

The implementation is a compact prototype rather than a hosted streaming service. Its strongest characteristics are clear separation between Flask routes, command logic, library discovery, and browser modules, together with a usable typed-command fallback. Important constraints include English-only browser speech recognition, browser-dependent media support, a small hand-authored training set, and the absence of automated test evidence in the repository. The report explains the architecture, requirements, implementation, privacy boundaries, evaluation approach, limitations, and practical improvements.

# TABLE OF CONTENTS

**1. Introduction**

**2. Background, Problem Definition and Objectives**

**3. Existing System and Proposed Approach**

**4. Scope and Use Cases**

**5. Requirements and Environment**

**6. System Architecture**

**7. Data Flow and Command Lifecycle**

**8. User Interface and Experience**

**9. Backend Implementation**

**10. Voice Interface and Intent Classification**

**11. Music Player and Personal Library**

**12. Optional Camera Expression Feature**

**13. Testing and Evaluation**

**14. Security, Privacy, and Reliability**

**15. Limitations and Future Enhancements**

**16. Conclusion**

**17.References**

# 1. Introduction

## 1.1 Project context

Music is often selected according to an activity or desired atmosphere. A listener may want calm music while resting, upbeat tracks while exercising, or low-distraction music while studying. Conventional players generally require the listener to search or navigate manually. VibeTune explores a lightweight interaction model in which users can issue a short voice or text request and continue controlling playback through familiar buttons.

## 1.2 Project overview

The repository implements a web application using Python and Flask on the server, HTML and CSS for presentation, and JavaScript for playback and interaction. The server scans four local categories—Calm, Happy, Focus, and Energy—and returns track metadata to the page. The browser supplies audio playback and, where supported, speech recognition and speech synthesis. A locally trained text classifier maps command examples to intent labels.

## 1.3 Report scope

This report is based on the public source tree and its README. It distinguishes implemented behavior from recommendations and does not claim formal user studies, measured accuracy, or production deployment. The application should be understood as a personal-library player that can be run locally, rather than a commercial music catalog. The remaining chapters explain the problem, goals, system environment, design, modules, user flows, validation considerations, privacy, and future work.

## 1.4 Intended use and report focus

The intended user is a listener who already has a collection of audio files and wants a quick way to choose among them. VibeTune does not search the internet for music or infer a user's personality. It translates an expressed request into a small set of available player actions and mood categories. The report therefore evaluates the project as a local interaction system: how the library is prepared, how a phrase travels through the browser and server, and how the resulting intent affects playback. The implementation is also considered in terms of usability, privacy boundaries, reliability, and the changes needed before wider deployment.

# 2. Background, Problem Definition and Objectives

## 2.1 Music discovery and interaction

A music library can become difficult to browse when the listener's intent is not expressed as a particular title. Mood-based organization offers a simple bridge between a broad preference and a set of tracks. Voice interaction can reduce navigation steps when the user is away from the keyboard, while typed commands provide the same intent-driven experience when microphone access is unavailable.

## 2.2 Natural-language intent

VibeTune uses a small supervised text-classification approach. Example phrases are paired with intent labels; TF-IDF features represent words and adjacent word pairs, and logistic regression estimates an intent from an input phrase. The implementation also applies explicit phrase rules for common transport and volume requests. This hybrid arrangement makes frequent commands predictable while allowing the classifier to handle varied mood wording.

## 2.3 Mood is not vocal emotion

The core command path interprets the words recognized by the browser. It does not analyze pitch, energy, or other acoustic qualities of the speaker's voice. A phrase such as 'I feel stressed' is treated as a request for the Calm category because of its text meaning. The optional camera path estimates facial expressions and is independent of voice-command classification. These distinctions are important for accurate product descriptions and user expectations.

## 2.4 Problem statement

Listeners may know the kind of music they want without knowing which specific track to select. Repeatedly searching, opening a playlist, and adjusting playback can be cumbersome. A small personal player can improve this interaction by accepting direct commands and organizing locally supplied tracks into understandable categories. The design challenge is to make these interactions useful without hiding the limits of recognition, browser support, or the available library.

## 2.5 Objectives

The project aims to provide a single-page player that accepts voice or typed English commands; recognizes common controls such as play, pause, next, previous, shuffle, repeat, and volume; maps mood wording to one of four music categories; discovers audio files from a predictable local folder structure; and offers essential browsing and personalization features. It also demonstrates a separable optional camera-based expression estimate.

### 2.6 Success criteria

A practical demonstration should show that the server starts, the page lists valid local tracks, typed commands reach the command endpoint, and playback controls operate when the browser supports the audio format. Voice input should be treated as an enhancement and tested only in a compatible browser with microphone permission. Mood selection should choose tracks from the corresponding category where tracks exist. The repository contains no benchmark results, so recognition quality must be evaluated empirically rather than assumed.

## 3. Existing System and Proposed Approach

### 3.1 Conventional interaction

A conventional local player usually relies on menus, search, and repeated pointer or keyboard actions. This interaction is reliable, but it can require a listener to remember where a track or playlist is stored. Keyword search is effective when the title is known, yet it does not directly represent a broad request such as music for concentration.

### 3.2 Proposed approach

VibeTune combines standard controls with intent-based commands. The listener may select a track from the visible library, type a request, or speak into the browser microphone. A command such as 'play something calm' is interpreted as a mood intent, after which the frontend selects an appropriate track from the available Calm category. If speech recognition is unsupported or permission is denied, the typed command box remains available.

### 3.3 Design trade-offs

The project favors local setup and simple components over remote catalog integration. Music files remain under the user's control and are not bundled. The command model can be retrained by adding examples, but its limited training vocabulary means it may not generalize to every phrasing. The camera feature offers another interaction option, but it adds model-download and permission requirements and should remain explicitly optional.

### 3.4 Comparison with the proposed player

The proposed player keeps manual browsing available and adds a lightweight command layer on top of

it. This is useful because recognition errors do not block the basic task: the listener can still select a track or use the playback buttons. Unlike a streaming catalog, the application depends on files supplied by the user, so its usefulness grows with the quality and size of that local library. Unlike a voice-only interface, it exposes the recognized request and gives visible confirmations. The design consequently treats voice as an alternate control method rather than the only way to operate the player.

# 4. Scope and Use Cases

## 4.1 In-scope capabilities

The implemented scope includes loading a browser interface, listing local music files in four categories, playing tracks, seeking, changing volume, navigating tracks, shuffle and repeat modes, favorites, custom playlists, library search and filters, theme switching, voice/text command processing, and a camera expression estimate with recommendations. The user can also use keyboard shortcuts and a Music Guide panel.

## 4.2 User scenarios

A listener can add audio files to a category folder and reload the page to see them. They can select a track, press play, adjust the timeline, or ask for another track. They can mark favorites and assemble a custom playlist. They may type 'music to study' and receive a Focus-oriented selection. On a supported browser, they may speak the same request. Separately, they can opt into a camera estimate and review recommendations.

## 4.3 Out of scope

The source does not implement user accounts, cloud synchronization, remote streaming catalogs, social sharing, a mobile-native application, multilingual speech recognition, or a production recommendation engine trained on listening behavior. It also does not claim clinically meaningful emotion recognition. These boundaries help set realistic expectations and inform future development priorities.

## 4.4 User workflow boundaries

A typical session begins with the user opening the local application and selecting a mood or track. The listener may then issue transport commands while playback continues, save a preferred track, or place tracks into a custom playlist. The system's responsibilities end at local discovery, command interpretation, and browser playback. It does not verify whether a file is licensed, repair damaged audio, or guarantee playback on every browser. Users should keep their files organized in the documented folders and choose formats supported by their browser. These assumptions keep the interaction

predictable and make troubleshooting easier.

# 5. Requirements and Environment

## 5.1 Software requirements

The README specifies Python 3.10 or newer. Dependencies are Flask 3.x and scikit-learn 1.4.x within the stated compatible ranges. The application runs with Python's development server through app.py. A current browser is required for the interface; browser support for audio codecs, the Web Speech API, speech synthesis, and camera access varies. The camera model assets are obtained from a public CDN when the feature is used.

## 5.2 Hardware and permissions

For basic playback, a computer with a web browser and audio output is sufficient. A microphone is needed only for voice input. A camera is needed only for the optional expression feature. The browser prompts for relevant permissions, and camera access generally requires localhost or HTTPS. Permission denial should not prevent typed commands or ordinary playback.

## 5.3 Operational assumptions

The application is intended to run locally from the project directory. It expects the static/music subfolders to exist and permits them to be populated by the user. File extensions are checked against a defined allowlist. Actual playback depends on browser decoding support. Since the app entry point enables Flask debug mode, the development server must not be exposed as a production service without configuration changes.

## 5.4 Nonfunctional requirements

The interface should remain usable when optional hardware or browser features are missing. Typed commands and visible controls are therefore required fallbacks for voice input, while camera recommendations remain optional. Responses should be clear when the library is empty, a command is unknown, or a browser blocks playback. Startup should be straightforward in a local development environment, and the library scan should use a fixed set of supported extensions. For deployment beyond a personal machine, security configuration, dependency maintenance, and a production server become additional requirements. These quality requirements complement the functional list by describing how the application should behave under ordinary constraints.

# 6. System Architecture

## 6.1 Architectural overview

The system follows a small client-server structure. Flask owns page rendering, static asset delivery, library discovery, and command classification. The browser owns interaction state, audio playback, speech input, rendering of the library, local personalization, and the optional face-expression estimation. The separation keeps media controls responsive while allowing the backend to supply consistent library metadata and intent decisions.

## 6.2 Request flow

When the browser requests the home page, the Flask route scans the mood directories and passes track metadata into the template. For a command, JavaScript sends JSON to POST /api/command. The route validates that text is present, includes up to five alternative speech transcripts if supplied, and classifies each candidate. The first recognized candidate is returned with intent, confidence, and transcript. The frontend then applies the intent to player state.

## 6.3 Component boundaries

The voxtune package contains the app factory, routes, command logic, and library discovery. Templates provide the page structure. Static JavaScript separates shared helpers, library management, player logic, optional camera behavior, and event wiring. CSS controls layout, responsive behavior, and themes. This arrangement is understandable for a small project and creates clear locations for future changes.

## 6.4 Component interaction

The app factory establishes the Flask application and registers the routes blueprint. The route layer obtains library data from the discovery function and supplies it to the HTML template. On the client, app.js connects page events and speech APIs to player.js, while library.js handles track display and saved collections. core.js provides shared state and user messages. camera.js contains the opt-in expression flow. This division means that a change to command examples can usually be made in the backend command module, while a change to playlist presentation belongs in the browser library module. The responsibilities are simple enough to trace during maintenance.

## 7. Data Flow and Command Lifecycle

### 7.1 Library data

The discovery function iterates through Calm, Happy, Focus, and Energy folders. It filters regular files by supported extension and derives a display title from the filename by removing common suffixes and cleanup markers. Each result includes a category, a display color, an artist label, and a URL rooted under /static/music. The README advises users to use music they own or have permission to play.

### 7.2 Command data

The browser obtains a transcript through speech recognition or accepts typed text. If speech recognition returns alternative transcripts, the frontend may pass them along with the primary phrase. The server normalizes text for rule checks, attempts explicit matches, then uses the trained pipeline when needed. The response contains a transcript selected for classification, a category-like intent name, and a rounded confidence score.

### 7.3 Playback action

The frontend maps the intent to a local action. Navigation, volume, shuffle, repeat, and transport commands update the HTML audio element or player state. Mood intents filter or select from the matching category. A named-track request is matched against available titles. Unknown or unsupported wording is designed to produce clarification feedback rather than an unrelated action. The interface presents a confirmation so the user can see what the system understood.

### 7.4 Error and fallback handling

The command endpoint rejects an empty phrase instead of attempting to classify it. When speech recognition supplies alternatives, the server limits the number it processes and uses the first candidate that is not classified as unknown. If no supported intent is found, the interface can ask the user to rephrase rather than executing a random action. Track discovery also has a natural empty-state condition: a category folder may contain no supported audio. At each boundary, the application should make the state visible and retain a direct control path. These behaviors reduce the impact of partial input, unavailable media, or uncertain recognition.

## 8. User Interface and Experience

### 8.1 Main screen

The page combines a prominent voice-command area, a visual music-themed hero, a now-playing card, and a library section. The player displays the current title, playback timeline, transport controls, volume, shuffle, and repeat. The library area supports browsing and can expose filters, favorites, playlist controls, and a surprise-me action. The layout is styled as a responsive personal music space.

### 8.2 Accessible alternatives

Voice controls are presented alongside a text input and clickable controls. The template uses button labels, status regions, and live announcements for key interface feedback. Keyboard shortcuts provide another route to common actions. These alternatives matter because microphones may be unavailable, speech APIs may not be supported, and some users prefer direct control.

### 8.3 Personalization

The frontend saves theme preference, favorites, and custom playlists in browser localStorage. This persistence is local to the browser profile and does not imply server-side accounts or synchronization. Stable track paths are used for custom playlist references so that adding files does not normally invalidate existing entries. Clearing browser storage can remove these preferences.

### 8.4 Feedback and usability

The interface presents status text near the microphone control and uses confirmations after commands so the listener can see the interpretation. The now-playing area provides a persistent indication of the selected track, while the timeline and transport buttons expose the current playback state. Search and category filters support users who prefer browsing. Keyboard shortcuts can shorten repeated actions, but they should not replace clearly labeled buttons. A polished implementation should also ensure that focus remains visible, dialogs can be closed by keyboard, motion can be reduced, and live status messages are concise. These details make the page more usable across input preferences.

# 9. Backend Implementation

## 9.1 Flask application factory

The package initializer defines create_app(), constructs a Flask application with project-level template and static directories, and registers the application blueprint. app.py imports the factory, creates the application object, and runs the development server when invoked as the main program. The factory keeps application setup concise and makes configuration extension possible.

## 9.2 Routes

The home route discovers the current library and renders index.html with those tracks. The command route accepts JSON, rejects empty text with a client error, gathers available titles, and classifies the primary phrase plus a bounded set of alternatives. It returns JSON containing the chosen transcript, intent, and confidence. This endpoint does not persist command history.

## 9.3 Library discovery

The library module centralizes category names, visual colors, and supported audio extensions. It scans each category directory and builds track records dynamically, so adding a supported audio file does not require editing the JavaScript track list. Filename cleanup improves readability but cannot reliably infer artist metadata; the current artist label is a generic library label.

## 9.4 Command endpoint behavior

The command endpoint is deliberately stateless: it receives a phrase, classifies it, and returns a result without recording a history or changing the player's audio directly. The frontend remains responsible for applying the intent, which keeps playback tied to the active browser session. Available track titles are passed into classification so named-track requests can be recognized against the current library. A production version should add explicit schema validation, request-size limits, structured error responses, and logging that avoids retaining unnecessary personal text. The current route is a compact demonstration endpoint and should be treated as such.

## 10. Voice Interface and Intent Classification

### 10.1 Browser speech recognition

The frontend uses SpeechRecognition or webkitSpeechRecognition where available and sets the recognition locale to en-US. It requests several alternatives and listens for a final result. Browser recognition availability and quality vary. Microphone errors update the interface, while typed commands remain usable. The project documentation correctly distinguishes browser recognition from a paid speech-recognition API.

### 10.2 Rule-based command handling

The classifier first applies explicit patterns for volume, mute, stop, pause, resume, shuffle, repeat, next, and previous. It then checks common mood words and named library titles. This ordering protects clear commands such as 'stop the song' from being interpreted as a mood request. Exact rules return a confidence of 1.0 as an implementation convention, not as a calibrated probability.

### 10.3 TF-IDF and logistic regression

The scikit-learn pipeline converts text into unigram and bigram TF-IDF features and fits logistic regression to examples in commands.py at import time. Intent categories include playback controls and four moods. A low-confidence result can become unknown. The phrase set is small and English-only, so ongoing improvements should add varied examples and evaluate performance against a held-out command set.

### 10.4 Recognition limits

The classifier's confidence value should be interpreted cautiously. Rule-based matches return a fixed high value, whereas the model's probability comes from a small training set and is not calibrated against a representative command population. A low threshold can help reject unrelated requests but may also accept an incorrect class. The best practical improvement is to collect varied, consented example phrases for each supported intent, separate evaluation examples from training data, and review confusion between similar categories. Recognition should also be tested with short phrases, background noise, different accents, and common transcription errors. Voice results remain dependent on the browser's speech service.

## 11. Music Player and Personal Library

### 11.1 Audio playback

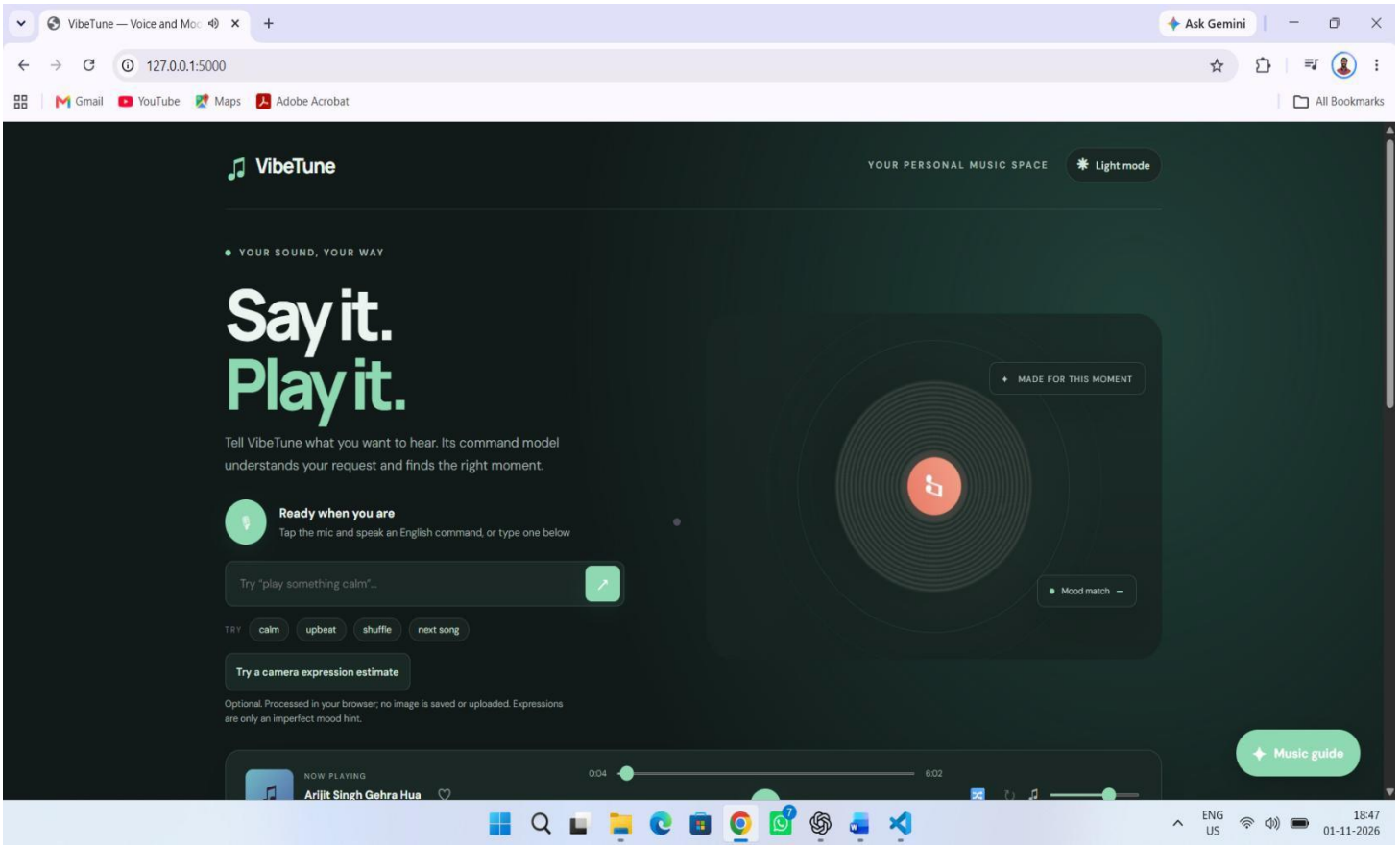
The browser's audio element is the playback engine. The interface wires play, pause, stop, next, previous, timeline seeking, and volume controls to the current track. Stop is documented as stopping and rewinding the active track. Browser autoplay policies may prevent automatic playback after a recommendation; the user can use the recommendation's Play button.

### 11.2 Mood folders

Four named directories organize the personal library. When a mood intent is selected, the player chooses from tracks associated with that category. The categories are intentionally simple and user-managed. A category with no files cannot provide a matching recommendation, which should be handled with a clear empty-state message. The repository's checked-in folders use placeholders rather than actual recordings.

### 11.3 Favorites and playlists

Favorites and custom playlists are stored in localStorage. Users can create a playlist, add the current track, and delete a selected playlist. Library search and filters help locate tracks. Since this state is browser-local, it is not available automatically on another device. Track metadata is derived largely from filenames, so richer tags or an editable metadata layer could improve organization.



## 12. Optional Camera Expression Feature

### 12.1 Feature flow

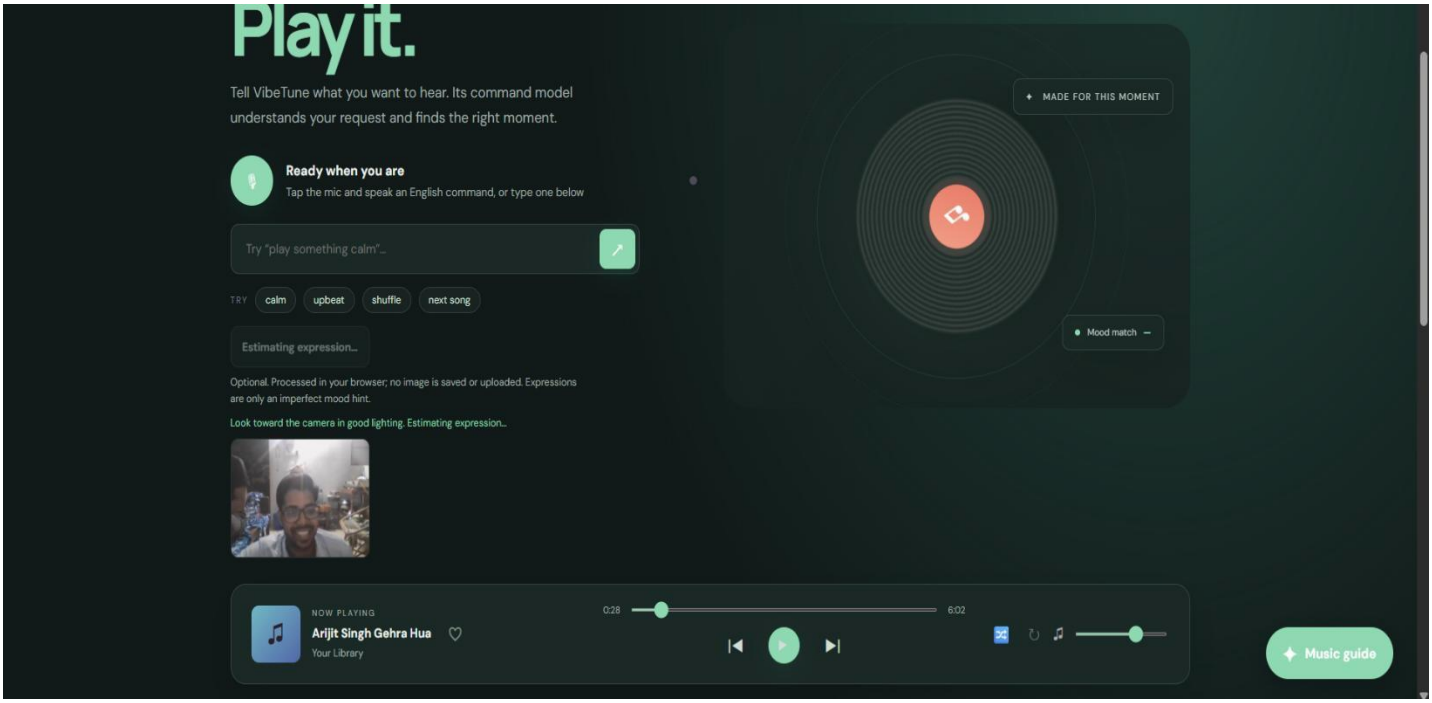
The camera feature is isolated in camera.js and begins only after the user activates the camera control and grants permission. The browser obtains a video stream, loads a face detector and expression model from a public model location, estimates an expression, maps the estimate to one of the player categories, and displays up to three recommended tracks. It attempts to play the first suggestion, subject to browser autoplay policy.

### 12.2 Processing and user choice

The README states that camera frames are not uploaded or saved by this application and that expression estimation occurs locally in the browser. The model files are downloaded from a public CDN on first use, so the feature has an external dependency even though frames are processed locally. Users can avoid this feature entirely and continue using typed or spoken text commands.

### 12.3 Interpretation limits

A facial-expression model estimates visible expression classes; it cannot establish a person's actual emotional state. Lighting, pose, image quality, and model limitations can affect results. The feature should be framed as an optional playful input for suggestions, not a psychological assessment. Clear permission prompts, an obvious stop path, and accessible non-camera alternatives are appropriate design requirements.



# 13. Testing and Evaluation

## 13.1 Validation plan

A functional review should cover application startup, page rendering, track discovery, filename cleanup, supported file extensions, empty categories, and every transport control. Command testing should include each explicit rule, each mood intent, named-track requests, spelling variants, unsupported phrases, empty API payloads, and alternative speech transcripts. Tests should confirm that a command selects the intended action rather than only returning a plausible label.

## 13.2 Browser and device coverage

Voice input should be checked in browsers that expose the Web Speech API, while typed-command behavior should be checked in all target browsers. Audio tests should use representative files in each supported format and account for codecs that a given browser cannot decode. Camera testing should cover permission granted, denied, unavailable hardware, model-download failure, no detected face, and autoplay rejection.

## 13.3 Evidence and metrics

The repository does not provide a test suite or measured model accuracy. A future evaluation can report intent accuracy, per-class precision and recall, unknown-command handling, task completion time, and user-reported clarity. Results should use a documented dataset of paraphrased commands separated from the training examples. Any reported values must come from actual runs and should identify browser and system conditions.

## 13.4 Acceptance criteria

A release candidate should pass a repeatable checklist on a clean local setup. At minimum, the server must start, the home page must load, supported tracks must appear in the right category, and playback controls must work for at least one tested format. Each command intent should be exercised with both a direct phrase and a paraphrase, and unknown input should not trigger an unrelated operation. Browser checks should record speech, audio, and camera support separately. Results should distinguish defects in the application from browser limitations. Keeping this checklist with the repository would make future changes easier to review and would turn the proposed evaluation into practical evidence.

## 14. Security, Privacy, and Reliability

### 14.1 Data handling

The core command endpoint receives text and returns a classification; the source does not store a command history. Browser speech recognition behavior is controlled partly by the browser and its provider. The README notes that the browser may process microphone audio according to its own provider and privacy settings. Therefore, the app should not promise that microphone audio never leaves the device.

### 14.2 Camera and local state

Camera frames are not uploaded or saved by the app according to its documentation, while model files are downloaded externally. Favorites, playlists, and theme preferences are stored in localStorage. These design choices reduce the need for an account backend, but users should understand the browser-local persistence and external model download. Permission should be requested only when the relevant control is activated.

### 14.3 Operational hardening

The app entry point runs Flask with debug mode enabled for local development. Before deployment, debug mode should be disabled, configuration should be separated by environment, dependencies should be pinned and reviewed, and the service should be placed behind a suitable production server. The command route should validate request types and size, and static music should be served only from the intended library paths.

### 14.4 Risk controls

The most relevant risks are accidental exposure of the development server, unclear microphone or camera expectations, and user confusion about where local preferences are stored. Debug mode should remain limited to development, while microphone and camera access should be requested only when the user activates the relevant control. The interface should accurately describe browser speech processing and the remote download of camera model files. LocalStorage should not be described as cloud backup. A future deployment should use a production WSGI server, restrict file access to intended static paths, keep dependencies current, and provide a privacy notice proportionate to the data the application handles.

# 15. Limitations and Future Enhancements

## 15.1 Current limitations

The command model is based on a compact hand-authored set and is limited to English. Browser recognition, microphone permission, audio codecs, autoplay policy, camera access, and CDN availability affect behavior. The project supplies no music catalog, user account system, analytics, or automated test suite. Mood folders are broad categories and do not represent a learned or personalized recommendation model.

## 15.2 Near-term improvements

Useful next steps include adding automated unit tests for command rules and library discovery, expanding training phrases, documenting an evaluation set, improving error and empty-library states, and adding editable metadata. A configuration option could disable camera loading entirely. Accessibility review should include keyboard-only use, screen-reader feedback, focus management, and reduced-motion preferences.

## 15.3 Longer-term direction

The project could support multilingual commands, import/export of playlists, improved metadata tagging, richer recommendation logic, and optional user-controlled synchronization. Such additions would require careful consent and data design. A recommendation engine should be evaluated on relevance and diversity and should explain the role of user-provided categories. Any remote music integration would need to respect the chosen provider's terms and copyright constraints.

## 15.4 Improvement priorities

The most effective development order is to strengthen reliability before adding a larger recommendation feature. First, add tests for library discovery, command rules, and API validation. Next, improve empty states and accessibility, then expand and measure the command vocabulary. Richer metadata and playlist import/export can follow once track identity is stable. Only after these foundations should the project consider accounts or synchronization, since those features introduce new privacy and security responsibilities. Each improvement should be evaluated against a clear user task, such as finding a track faster or recovering a playlist, rather than added solely to increase feature count.

# 16. Conclusion

## 16.1 Summary

VibeTune demonstrates how a local music library can be combined with browser-based voice input, text command classification, and standard playback controls. Flask provides the page and command endpoint; scikit-learn classifies text; JavaScript manages the player, library, and optional camera path. Users provide their own audio files, organized into Calm, Happy, Focus, and Energy folders.

## 16.2 Project assessment

The implementation is strongest as a compact learning project and local prototype. It offers several interaction paths and keeps major responsibilities in separate modules. Its documentation is particularly useful in clarifying that spoken words are classified as text, that camera expression estimates can be inaccurate, and that browser support varies. These qualifications are essential to presenting the system accurately.

## 16.3 Closing remarks

With test coverage, broader command examples, accessible error handling, and deployment hardening, the project could become a more dependable personal player. Future additions should preserve user control over microphone and camera use, avoid unsupported emotional claims, and make recommendation behavior transparent. The current foundation provides a clear starting point for those improvements.

## 16.4 Final assessment

VibeTune meets its core educational objective by joining a local Flask application, text-based intent classification, browser speech input, and a modular JavaScript player. It demonstrates practical trade-offs: local music gives users control but requires setup; voice commands reduce navigation but depend on browser recognition; camera suggestions add an alternate input but require explicit consent and cautious interpretation. The project is most credible when described within these boundaries. With automated checks, improved error handling, and deployment hardening, it can provide a more dependable local listening experience while keeping direct controls available to every user.

## References


1. H. Tran, T. Le, A. Do, T. Vu, S. Bogaerts, and B. Howard, "Emotion-Aware Music Recommendation," *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 37, no. 13, pp. 16087–16095, 2024. DOI: 10.1609/aaai.v37i13.26911.
2. "Smart Music Recommendation System Integrating NLP and Hybrid Algorithms," *2024 2nd International Conference on Advances in Computation, Communication and Information Technology (ICAICCIT)*, IEEE, 2024. DOI: 10.1109/ICAICCIT64383.2024.10912332.
3. "Emotion Based Music Recommendation System using Machine Learning," *2024 International Conference on IoT, Communication and Automation Technology (ICICAT)*, IEEE, 2024. DOI: 10.1109/ICICAT62666.2024.10923478.
4. "Song Recommendation System based on Mood Detection using Spotify's Web API," *2022 International Interdisciplinary Humanitarian Conference for Sustainability (IIHC)*, IEEE, 2022. DOI: 10.1109/IIHC55949.2022.10060806.
5. D. Ayata, Y. Yaslan, and M. E. Kamaşak, "Emotion Based Music Recommendation System Using Wearable Physiological Sensors," *IEEE Transactions on Consumer Electronics*, 2018. DOI: 10.1109/TCE.2018.2844736.
6. "Emotion Recognition and Music Recommendation System based on Facial Expression," *2024 Second International Conference on Advances in Information Technology (ICAIT)*, IEEE, 2024. DOI: 10.1109/ICAIT61638.2024.10690441.
7. J. S. Gómez-Cañón et al., "TROMPA-MER: An Open Dataset for Personalized Music Emotion Recognition," *Journal of Intelligent Information Systems*, vol. 60, pp. 549–570, 2023. DOI: 10.1007/s10844-022-00746-0.
8. J. Kang and D. Herremans, "Are We There Yet? A Brief Survey of Music Emotion Prediction Datasets, Models and Outstanding Challenges," *IEEE Transactions on Affective Computing*, vol. 16, no. 4, pp. 2545–2559, 2025. DOI: 10.1109/TAFFC.2025.3583505.
9. "Real-Time Speech Emotion Recognition Using Deep Learning for Emotion Based Music Recommendation," *7th IEEE International Conference on Artificial Intelligence in Engineering and Technology (IICAIET)*, 2025, pp. 754–759. DOI: 10.1109/IICAIET67254.2025.11265611.
10. "Emotion-Based Music Recommendation System Using Deep Learning," *2025 3rd IEEE International Conference on Industrial Electronics: Developments & Applications (ICIDeA)*, IEEE, 2025. DOI: 10.1109/ICIDeA64800.2025.10963221.

## Appendix

## ASSESSMENT

**Internal:**

| SI. NO. | RUBRICS | FULL MARKS | MARKS OBTAINED | REMARKS |
|---|---|---|---|---|
| 1 | Understanding the relevance, scope and dimension of the project | 10 | | |
| 2 | Methodology | 10 | | |
| 3 | Quality of Analysis and Result | 10 | | |
| 4 | Interpretation and Conclusion | 10 | | |
| 5 | Report | 10 | | |
| | **Total** | **50** | | |

**Date:** **Signature of the Faculty**